农产品溯源系统 - 技术文档

一、系统概述

本系统是一套基于区块链技术的农产品全链路溯源平台，覆盖从种植/养殖、检疫质检、加工宰杀、仓储、物流运输到销售的完整供应链流程。系统由三端组成：PC 管理后台（Vue.js）、微信小程序（原生框架）、Spring Boot 后端服务，并集成以太坊区块链（Ganache 本地测试网）实现数据不可篡改存证。

二、技术栈

后端

- Spring Boot 2.7.18 + Java 11
- MyBatis Plus 3.5.3.1（ORM 框架，逻辑删除、分页插件）
- MySQL 8.0 + Druid 连接池
- Web3j 4.9.8（以太坊 Java SDK）
- JWT（jjwt 0.11.5，无状态鉴权）
- Hutool 5.8.23（工具库）
- Spring AOP（操作日志切面）
- Spring Security Crypto（密码加密）

小程序

- 微信原生小程序框架（WXML + WXSS + JS）
- 统一请求封装（utils/request.js）
- 环境配置（config.js，支持 dev/prod 切换）

区块链

- Ganache 2.7+（本地以太坊测试网，RPC: http://127.0.0.1:8545）
- Solidity 0.8.x 智能合约（Traceability.sol）
- Remix IDE（合约编译与部署）

三、项目目录结构

```
ProductTraceability_2026/
├── 01_Database/
|   └── agricultural_database.sql          # 完整数据库建表+初始化脚本
|
├── 02_Backend/                            # Spring Boot 后端（端口 8888）
|   ├── pom.xml
|   ├── uploads/                           # 文件上传存储目录（运行时生成）
|   └── src/main/
|       ├── resources/
```

```
|       |   └── application.yml            # 数据库/区块链/文件上传等核心配置
|       └── java/com/agricultural/trace/
|           ├── AgriculturalTraceApplication.java   # 启动类
|           ├── annotation/
|           |   └── OperationLog.java               # 操作日志自定义注解
|           ├── aspect/
|           |   └── OperationLogAspect.java          # AOP 切面，自动记录操作日志
|           ├── common/
|           |   └── Result.java                     # 统一响应体 Result<T>
|           ├── config/
|           |   ├── BlockchainConfig.java            # 区块链参数配置
|           |   ├── CorsConfig.java                  # 跨域配置
|           |   ├── FileUploadConfig.java            # 文件上传路径配置
|           |   ├── JacksonConfig.java               # JSON 序列化配置
|           |   ├── MyBatisPlusConfig.java           # 分页插件配置
|           |   └── OperationLogSchemaInitializer.java  # 操作日志表初始化
|           ├── controller/                # 控制器层（28个，对应各业务模块）
|           |   ├── AuthController.java              # /auth 认证登录
|           |   ├── EnterpriseController.java        # /api/enterprise 企业管理
|           |   ├── EnterpriseAuditController.java   # /api/enterprise/audit 企业
审核
|           |   ├── BatchController.java             # /api/batch 批次管理
|           |   ├── RecordController.java            # /api/record 过程记录
|           |   ├── QuarantineController.java        # /api/quarantine 检疫质检
|           |   ├── ProcessingController.java        # /api/processing 加工记录
|           |   ├── StorageController.java           # /api/storage 仓储管理
|           |   ├── TransportController.java         # /api/transport 运输管理
|           |   ├── SaleController.java              # /api/sale 销售记录
|           |   ├── TraceController.java             # /api/trace 溯源查询
|           |   ├── BlockchainController.java        # /api/blockchain 区块链统计
|           |   ├── BlockchainContractController.java # /api/blockchain/contract
合约管理
|           |   ├── DashboardController.java         # /api/dashboard 数据看板
|           |   ├── SystemUserController.java        # /api/system/user 用户管理
|           |   ├── SystemRoleController.java        # /api/system/role 角色管理
|           |   ├── SystemMenuController.java        # /api/system/menu 菜单管理
|           |   ├── SystemConfigController.java      # /api/system/config 系统配置
|           |   ├── SystemLogController.java         # /api/system/log 日志查询
|           |   ├── FileController.java              # /api/file 文件上传
|           |   ├── BannerController.java            # /api/banner 轮播图
|           |   ├── EduArticleController.java        # /api/edu/article 科普文章
|           |   ├── MessageController.java           # /api/message 系统消息
|           |   ├── FeedbackController.java          # /api/feedback 用户反馈
|           |   ├── ImageAuditController.java        # /api/image-audit 图片审核
|           |   ├── ScanStatsController.java         # /api/scan 扫码统计
|           |   ├── TemplateController.java          # /api/template 溯源模板
|           |   └── DebugController.java             # 调试接口（开发环境）
|           ├── dto/                       # 请求入参对象
|           |   ├── LoginDTO.java
|           |   ├── BatchQueryDTO.java
|           |   ├── BatchReceiveDTO.java
|           |   ├── EnterpriseAuditDTO.java
|           |   └── QuarantineSubmitDTO.java
|           ├── vo/                        # 响应视图对象
|           |   ├── LoginVO.java
|           |   ├── UserInfoVO.java
```

```
|           |   ├── BatchVO.java
|           |   └── EnterpriseAuditVO.java
|           ├── entity/                    # 数据库实体（25个，与数据库表一一对应）
|           |   ├── SysUser / SysRole / SysMenu / SysConfig ...
|           |   ├── EnterpriseInfo / EnterpriseAuditImage
|           |   ├── TraceBatch / TraceRecord / TraceInspection ...
|           |   └── BlockchainTransaction / BlockchainContract ...
|           ├── mapper/                    # MyBatis Mapper 接口（与 entity 一一对
应）
|           ├── service/                   # 业务逻辑层（28个）
|           |   ├── AuthService.java                # 登录/注册/Token验证
|           |   ├── BatchService.java               # 批次创建/状态流转/上链
|           |   ├── QuarantineService.java          # 检疫申请/结果提交/上链
|           |   ├── TraceService.java               # 溯源详情聚合/区块链验真
|           |   ├── BlockchainTransactionService.java # 链上交易记录/Gas统计
|           |   ├── BlockchainNodeService.java       # 以太坊节点状态检测
|           |   ├── MessageService.java              # 系统消息发送/查询
|           |   ├── AdminBusinessNotificationService.java  # 业务事件通知
|           |   └── ...（其余 Service 与 Controller 一一对应）
|           ├── utils/
|           |   ├── Web3jUtils.java                  # 区块链上链/验证核心工具
|           |   ├── JwtUtils.java                    # JWT 生成/解析（有效期7天）
|           |   └── DataSeeder.java                  # 演示数据注入（seed profile）
|           ├── util/
|           |   └── FileCleanupUtil.java             # 文件清理工具
|           ├── task/
|           |   └── FileCleanupTask.java             # 定时清理无效上传文件
|           └── exception/
|               └── GlobalExceptionHandler.java      # 全局异常处理
|
├── 03_Frontend_Web/                      # PC 管理后台（Vue 2 + Element UI，端口
8080）
|   ├── package.json
|   ├── vue.config.js
|   ├── .env.development                  # 开发环境后端地址
|   ├── .env.production                   # 生产环境后端地址
|   └── src/
|       ├── main.js                       # 应用入口
|       ├── App.vue
|       ├── permission.js                 # 路由权限守卫
|       ├── api/                          # 后端接口封装（14个模块）
|       |   ├── user.js / system.js / enterprise.js / batch.js
|       |   ├── quarantine.js / record.js / storage.js / transport.js / sale.js
|       |   ├── trace.js / dashboard.js / template.js / scanStats.js / message.js
|       ├── router/
|       |   └── index.js                  # 路由配置（含动态路由）
|       ├── store/
|       |   ├── index.js
|       |   └── modules/
|       |       ├── user.js               # 用户状态（Token/用户信息）
|       |       └── permission.js         # 动态菜单权限
|       ├── layout/                       # 整体布局框架
|       |   └── components/
|       |       ├── Navbar.vue            # 顶部导航栏
|       |       ├── Sidebar.vue           # 左侧菜单栏
|       |       └── AppMain.vue           # 主内容区
```

```
|       ├── utils/
|       |   ├── request.js                # Axios 封装（自动携带 Token/401处理）
|       |   └── auth.js                   # Token 本地存储工具
|       └── views/                        # 页面视图（按功能模块分目录）
|           ├── login/          # 登录、企业注册
|           ├── dashboard/      # 首页数据看板
|           ├── enterprise/     # 企业入驻审核、图片审核、企业列表
|           ├── enterprise-self/ # 企业自身信息管理
|           ├── production/     # 批次初始化、过程记录（种植养殖端）
|           ├── circulation/    # 检疫申请/结果、仓储、运输、销售（流通端）
|           ├── processing/     # 加工记录（加工端）
|           ├── blockchain/     # 区块链数据统计、节点监控、合约管理
|           ├── chain/          # 手动上链操作
|           ├── supervision/    # 溯源数据查询、数据完整性检查
|           ├── statistics/     # 平台统计大屏、扫码统计
|           ├── template/       # 溯源模板管理
|           ├── content/        # 科普文章管理
|           ├── analysis/       # 数据分析统计
|           └── system/         # 用户管理、轮播图、系统配置、日志、消息
|
├── 04_Frontend_MiniApp/                  # 微信小程序（原生框架）
|   ├── app.js                            # 全局逻辑（登录状态/Token管理）
|   ├── app.json                          # 全局配置（TabBar/页面注册）
|   ├── app.wxss                          # 全局样式
|   ├── config.js                         # 环境配置（dev/prod API地址切换）
|   ├── project.config.json               # 小程序项目配置（AppID等）
|   ├── sitemap.json
|   ├── images/                           # 静态图片资源
|   ├── utils/
|   |   └── request.js                    # 请求封装（自动携带Token/401处理）
|   └── pages/                            # 所有页面（每页4个文件：
js/json/wxml/wxss）
|       ├── index/          # 首页（平台宣传/企业视图）、企业列表、企业详情
|       ├── scan/           # 扫码页（调用 wx.scanCode）
|       ├── trace/          # 溯源详情（全生命周期时间轴+区块链验真）
|       ├── workbench/      # 工作台（按角色展示不同功能入口）
|       ├── batch/          # 批次列表、创建批次、批次详情、过程记录
|       ├── collect/        # 数据采集（过程/加工/检疫/仓储/运输/销售/接收）
|       ├── quarantine/     # 检疫申报列表、结果录入、报告上传
|       ├── blockchain/     # 存证证书、上链记录列表
|       ├── education/      # 科普文章列表
|       ├── enterprise/     # 企业公示、审核详情、资质证书
|       ├── admin/          # 管理员专属（企业管理、用户管理、区块链管理）
|       ├── user/           # 个人中心、企业入驻注册、企业信息管理、消息、修改密码
|       └── login/          # 登录、注册
|
└── 05_SmartContract/                     # 以太坊智能合约
    ├── Traceability.sol                  # 溯源哈希存证合约（Solidity 0.8.x）
    ├── contract_abi.json                 # 合约 ABI（后端 Web3j 调用所需）
    └── README.md                         # 合约部署说明
```

四、数据库设计

数据库名：`agricultural`，字符集：`utf8mb4_unicode_ci`

4.1 系统管理表

表名	说明
sys_user	系统用户表。字段：id、username、password（BCrypt加密）、real_name、nickname、phone、email、avatar、user_type（1-管理员 2-企业用户 3-普通用户）、enterprise_id、enterprise_type（1-种植养殖 2-加工宰杀 3-检疫质检，与enterprise_info同步）、status、last_login_time、login_count
sys_role	角色表。字段：id、role_code、role_name、role_type（1-平台管理员 2-种植养殖 3-加工宰杀 4-检疫质检）、sort、status
sys_user_role	用户角色关联表（多对多）
sys_role_menu	角色菜单关联表（多对多）
sys_menu	菜单权限表。字段：id、parent_id、menu_name、menu_type（目录/菜单/按钮）、path、component、permission、icon、sort、status
sys_config	系统配置表。字段：config_key、config_value、config_name、remark
sys_login_log	登录日志表。字段：id、user_id、username、ip、location、browser、os、login_time、status、message
sys_operation_log	操作日志表。字段：id、user_id、username、module、operation、method、params、result、ip、location、browser、os、status、error_msg、execute_time、create_time
sys_message	系统消息表。字段：id、user_id、type（system/business）、title、summary、content、is_read、action_url、action_title、action_desc、create_time
sys_feedback	用户反馈表。字段：id、user_id、content、status（0-待处理 1-已处理）、reply、reply_time、reply_by、create_time
sys_banner	轮播图表。字段：id、enterprise_id（NULL表示平台级通用）、title、description、image_url、link_url、target_type（0-所有人/游客 1-种植养殖企业 2-加工宰杀企业 3-检疫质检企业）、sort_order、status
sys_dict_type	数据字典类型表。字段：id、dict_name、dict_type（唯一）、status
sys_dict_data	数据字典数据表。字段：id、dict_type、dict_label、dict_value、dict_sort、css_class、list_class、is_default、status

4.2 企业管理表

表名	说明
enterprise_info	企业信息表。字段：id、enterprise_code（唯一）、enterprise_name、enterprise_type（1-种植养殖 2-加工宰杀 3-检疫质检）、legal_person、contact_person、contact_phone、contact_email、province/city/district/address、business_license（营业执照URL）、production_license（生产许可证URL）、other_certificates（JSON）、introduction、logo、cover_image、wallet_address（以太坊地址）、audit_status（0-待审核 1-已通过 2-已拒绝）、audit_remark、audit_time、status
enterprise_audit_image	企业图片审核表。字段：id、enterprise_id、user_id、field_name（logo/cover_image等）、old_value、new_value、audit_status、audit_remark、create_time

4.3 溯源模板表

表名	说明
trace_template	溯源模板表。字段：id、template_code、template_name、template_type（1-养殖 2-种植）、product_category、config_json、sort、status
trace_stage	溯源阶段表。字段：id、template_id、stage_code、stage_name、stage_type（1-初始化 2-养殖记录 3-检疫 4-加工 5-仓储 6-运输 7-销售）、sort、is_required
trace_field	溯源字段表。字段：id、stage_id、field_code、field_name、field_type（text/number/date/select/image/video）、field_options（JSON）、is_required、sort、placeholder、default_value、validation_rule

4.4 核心溯源业务表

表名	说明
trace_batch	批次主表。字段：id、batch_code（唯一，格式 BATCH+时间戳）、product_name、product_type（1-养殖 2-种植）、breed（品种）、enterprise_id、init_quantity/current_quantity/unit、origin_location、latitude/longitude（GPS）、plant_area/greenhouse_no（种植专属）、seed_source、manager、production_date、expected_harvest_date、actual_harvest_date、batch_status（1-初始化 2-生长中 3-已收获/待检疫 4-加工中 5-已检疫 6-已入库 7-运输中 8-已销售 9-加工完成）、tx_hash、block_number、chain_time、data_hash（SHA-256）、receiver、receive_date、receive_enterprise_id
trace_record	养殖/种植过程记录表。字段：id、batch_id、batch_code、record_type（feeding-喂养 vaccine-防疫 inspect-巡查 fertilize-施肥 irrigate-灌溉 pesticide-除虫）、record_date、item_name、amount、description、operator、images（JSON数组）、location、latitude/longitude、tx_hash、block_number、chain_time、data_hash、deleted（软删除）

表名	说明
trace_inspection	检疫质检记录表。字段：id、batch_id、batch_code、inspection_date、check_result（1-合格 0-不合格 NULL-待审核）、inspection_items、cert_no（证书编号）、cert_image（JSON数组）、inspector、inspector_code、inspector_id、inspection_enterprise_id、tx_hash、block_number、chain_time、data_hash
trace_processing	加工宰杀记录表。字段：id、batch_id、source_batch_code、processing_enterprise_id、processing_date、process_method（屠宰/分割/包装）、specs（规格）、operator、input_quantity/input_unit、output_quantity/output_unit、images（JSON）、tx_hash、block_number、chain_time、data_hash
trace_storage	仓储记录表。字段：id、batch_id、batch_code、storage_enterprise_id、storage_date、warehouse_name、warehouse_location、temperature、humidity、operator、images（JSON）、tx_hash、block_number、chain_time、data_hash
trace_transport	运输记录表。字段：id、batch_id、batch_code、transport_enterprise_id、transport_date、departure/destination、vehicle_no、driver_name、driver_phone、temperature、operator、images（JSON）、tx_hash、block_number、chain_time、data_hash
trace_sale	销售记录表。字段：id、batch_id、batch_code、sale_enterprise_id、sale_date、sale_location、buyer_name、sale_quantity/sale_unit、sale_price、operator、images（JSON）、tx_hash、block_number、chain_time、data_hash

4.5 区块链相关表

表名	说明
blockchain_transaction	链上交易记录表。字段：id、tx_hash（唯一）、message_id、business_type（1-批次 2-记录 3-检疫 4-加工 5-仓储 6-运输 7-销售）、business_id、batch_id、data_hash、contract_address、status、block_number、block_hash、from_address、gas_used、gas_price、transaction_fee、chain_time
blockchain_gas_fee	日Gas费用统计表。字段：id、stat_date、enterprise_id、transaction_count、total_gas_used、total_fee_wei、total_fee_eth、avg_gas_price
blockchain_contract	智能合约信息表。字段：id、contract_address、contract_name、abi_json、deploy_time、status

4.6 其他业务表

表名	说明
edu_article	科普教育文章表。字段：id、enterprise_id（NULL表示平台级）、category（farming/vaccine/process/hygiene/quarantine/blockchain/safety）、title、author、summary、cover_url、content（JSON格式正文）、view_count、sort_order、status（0-草稿 1-已发布）、publish_time
user_scan_log	扫码统计表。字段：id、batch_code、batch_id、openid（微信OpenID）、scan_time、scan_location、ip_address、device_type、user_agent

---

五、后端模块说明

后端服务监听端口 **8888**，所有业务接口统一前缀 `/api`，认证接口前缀 `/auth`。统一响应格式为 `Result<T>{ code, message, data }`。

5.1 认证模块（AuthController）

路径前缀：`/auth`

方法	路径	功能
POST	/auth/login	用户名密码登录，返回 JWT Token 和用户信息，同时写入登录日志
POST	/auth/register	普通用户注册（消费者账号，user_type=3）
GET	/auth/info	根据 Token 获取当前用户信息（Token 有效性验证）
POST	/auth/logout	退出登录（前端清除 Token，后端无状态，无需处理）

AuthService 核心逻辑：

`login(LoginDTO)` 执行步骤：

1. 按 username 查询 sys_user，若不存在或 status=0 则返回错误
2. 使用 BCrypt 比对密码，不匹配则返回错误
3. 生成 JWT Token（有效期 7 天，携带 userId 和 username），通过 `Authorization: Bearer <token>` 请求头传递
4. 查询关联的 enterprise_info（若 enterprise_id 不为空），将 enterprise_type、audit_status 一并返回
5. 写入 sys_login_log（登录时间、IP、设备信息）

`registerOrdinaryUser(body)`：创建 user_type=3 的普通用户，密码 BCrypt 加密，不绑定企业。

5.2 企业管理模块（EnterpriseController + EnterpriseAuditController）

EnterpriseController 路径前缀：`/api/enterprise`

方法	路径	功能
POST	/api/enterprise/register	企业入驻注册（创建 enterprise_info + 关联/创建 sys_user，发送审核通知给管理员）
GET	/api/enterprise/list	分页查询企业列表（支持按名称/关键词/类型/审核状态筛选）
GET	/api/enterprise/list-by-type	按企业类型查询已审核通过的企业列表（供检疫申请时选择检疫机构）
GET	/api/enterprise/detail/{id}	查询企业详情（含上链数、批次数、溯源验证次数、最后上链时间）
GET	/api/enterprise/metrics/{id}	查询企业统计指标
PUT	/api/enterprise/update	更新企业信息（Logo/封面图变更自动触发图片审核流程）
PUT	/api/enterprise/toggle-status/{id}	启用/禁用企业
PUT	/api/enterprise/reapply/{id}	重新提交被驳回的入驻申请（audit_status 重置为 0）
DELETE	/api/enterprise/delete-rejected/{id}	删除被驳回的入驻申请

EnterpriseAuditController 路径前缀：`/api/enterprise/audit`

方法	路径	功能
GET	/api/enterprise/audit/list	分页查询待审核企业列表（audit_status=0）
GET	/api/enterprise/audit/detail/{id}	查询审核详情
POST	/api/enterprise/audit/approve	审核企业（通过/驳回），通过后自动更新关联用户权限
POST	/api/enterprise/audit/batch-approve	批量审核

审核通过时系统自动执行（同一事务）：

1. 更新 enterprise_info.audit_status = 1
2. 更新 sys_user.user_type = 2、enterprise_id = 企业ID、enterprise_type = 企业类型
3. 在 sys_user_role 中插入对应角色关联记录
4. 发送审核通过消息通知给申请人

审核驳回时： 更新 audit_status = 2，记录驳回原因，发送驳回通知。

5.3 批次管理模块（BatchController）

路径前缀：`/api/batch`

方法	路径	功能
GET	/api/batch/list	分页查询批次列表（支持多状态、关键词、企业类型、是否上链等多维筛选）
GET	/api/batch/status-count	查询各状态批次数量统计
GET	/api/batch/detail	查询批次详情（支持 batchCode 或 id 两种方式，batchCode 优先）
POST	/api/batch/create	创建新批次（同时支持 /add 路径），创建后自动上链存证
DELETE	/api/batch/delete/{id}	删除批次（仅允许状态为 1-初始化 或 4-加工中的批次）
GET	/api/batch/quarantine-check/{batchCode}	查询批次检疫合格状态（加工企业扫码前调用）
POST	/api/batch/receive	扫码接收批次（前置条件：必须有 check_result=1 的检疫合格记录）
POST	/api/batch/cancel-receive/{batchId}	取消接收批次（状态回滚为 3）
POST	/api/batch/reject/{batchId}	拒绝接收批次（状态回滚为 6，删除对应运输记录，通知养殖企业）

批次状态流转（完整）：

```
1(初始化) -> 2(生长中) -> 3(已收获/待检疫)
                                |
                          [提交检疫] -> check_result=1 -> 5(已检疫)
                                                              |
                                                     [加工企业接收] -> 4(加工中)
                                                                          |
                                                                  [创建加工记录] ->
9(加工完成)

    |

[入库] -> 6(已入库)

         |

   [运输] -> 7(运输中)

              |

        [销售] -> 8(已销售)
```

状态说明：

- 1：初始化（批次刚创建）
- 2：生长中（有过程记录后）
- 3：已收获/待检疫
- 4：加工中（加工企业已接收）
- 5：已检疫（检疫合格）
- 6：已入库
- 7：运输中
- 8：已销售
- 9：加工完成（ProcessingService.createProcessing 执行后自动设置）

BatchService 核心逻辑：

`createBatch(params)`：

1. 优先使用前端传入的 batchCode，否则自动生成（`BATCH` + yyyyMMddHHmmss + 4位随机数）
2. 插入 trace_batch，初始 batch_status=1
3. 生成二维码 URL：`http://api.domain.com/trace?code={batchCode}`
4. 计算 SHA-256 哈希（`BATCH_INIT|code=...|product=...|qty=...|enterprise=...|ts=...`），traceId 格式为 `{batchCode}_INIT`
5. 调用 Web3jUtils.uploadHash() 上链，保存 tx_hash、block_number、chain_time
6. 发送消息通知给所有管理员

`receiveBatch(dto)`：

1. 校验批次状态（必须为 3/5/6/7 之一）
2. 校验 trace_inspection 中存在 check_result=1 的记录，否则拒绝

3. 更新 batch_status=4，写入 receiver、receive_date、receive_enterprise_id

`rejectBatch(batchId, reason, rejectEnterpriseId)`：

1. 校验批次状态必须为 7（运输中）
2. 回滚 batch_status=6（已入库）
3. 删除指向该加工企业的 trace_transport 记录
4. 向养殖企业发送业务消息通知

状态统计（getStatusCount）：

- 养殖企业视角：breeding（状态1+2）、pending（状态3）、completed（状态4-9）
- 加工企业视角：pendingReceive（状态5+7，目标为本企业）、processing（状态4+9）、completed（状态6+8+9）

5.4 溯源记录模块（RecordController）

路径前缀：`/api/record`

方法	路径	功能
GET	/api/record/list	分页查询生长/种植过程记录（按 batchId、recordType 筛选）
POST	/api/record/create	新增过程记录（支持通过 batchCode 查找批次），创建后自动上链
DELETE	/api/record/delete/{id}	软删除记录（设置 deleted=1，物理文件由定时任务 7 天后清理）

记录类型（record_type）：feeding（喂养/施肥）、vaccine（防疫/用药）、inspect（巡查）、fertilize（施肥）、irrigate（灌溉）、pesticide（除虫）

上链 traceId 格式：`{batchCode}_RECORD_{recordId}`

5.5 检疫质检模块（QuarantineController）

路径前缀：`/api/quarantine`

方法	路径	功能
POST	/api/quarantine/apply	提交检疫申请（养殖企业发起，通知管理员和检疫机构）
DELETE	/api/quarantine/apply/delete/{id}	删除检疫申请（待检疫记录物理删除；已完成记录回滚 check_result 并将批次状态回退为 3）
GET	/api/quarantine/apply/list	查询检疫申请列表（养殖企业视角）
GET	/api/quarantine/apply/list-for-inspector	查询待处理/已完成检疫列表（检疫机构视角，按 inspection_enterprise_id 过滤）

方法	路径	功能
POST	/api/quarantine/submit	提交检疫结果（检疫机构填写结果、证书编号、证书图片，自动上链）
POST	/api/quarantine/update-certificate	更新检疫证书信息

QuarantineService 核心逻辑：

`applyQuarantine(params)`：

1. 查找批次（batchCode 或 batchId）
2. 创建 trace_inspection 记录，check_result=null（待检疫状态）
3. 若未指定 inspection_enterprise_id，自动查找 enterprise_type=3 且已审核通过的第一家检疫机构
4. 更新 batch_status=3（已收获/待检疫）
5. 本步骤不上链，上链在 submitQuarantine 时执行

`submitQuarantine(dto)`：

1. 查找该批次 check_result=null 的待检疫记录，有则更新，无则新增
2. 填写 check_result（1-合格/0-不合格）、inspector、cert_no、cert_image
3. 检疫合格时：更新 batch_status=5（已检疫）
4. 调用 performChainUpload() 上链

`performChainUpload(inspection, batch)`：

1. 构建哈希字符串（包含批次信息 + 检疫结果 + 证书信息），计算 SHA-256
2. traceId 格式：`{batchCode}_INSPECTION_{inspectionId}`
3. 调用 Web3jUtils.uploadHash() 上链，保存 tx_hash、block_number、chain_time
4. 调用 BlockchainTransactionService.recordTransaction() 记录交易
5. 发送上链成功消息通知给检疫机构

5.6 加工记录模块（ProcessingController）

路径前缀：`/api/processing`

方法	路径	功能
POST	/api/processing/create	创建加工记录（屠宰/分割/包装），自动上链，更新 batch_status=9（加工完成）
PUT	/api/processing/update	更新加工记录
DELETE	/api/processing/delete/{id}	删除加工记录
GET	/api/processing/list	分页查询加工记录（支持 enterpriseId、keyword 筛选）

上链 traceId 格式：`{batchCode}_PROC_{processingId}`

加工完成后，output_quantity 字段记录产出数量，供仓储录入页自动填入。

5.7 仓储管理模块（StorageController）

路径前缀：`/api/storage`

方法	路径	功能
GET	/api/storage/available-quantity	查询批次可入库数量（加工产出数量 - 已入库数量）
GET	/api/storage/list	分页查询仓储记录
POST	/api/storage/create	创建入库记录（记录仓库名称、位置、温湿度），自动上链，更新 batch_status=6
PUT	/api/storage/update	更新仓储记录
DELETE	/api/storage/delete/{id}	删除仓储记录

上链 traceId 格式：`{batchCode}_STORAGE_{storageId}`

5.8 运输管理模块（TransportController）

路径前缀：`/api/transport`

方法	路径	功能
GET	/api/transport/available-quantity	查询批次可运输数量（已入库数量 - 已运输数量）
GET	/api/transport/list	分页查询运输记录
POST	/api/transport/create	创建运输记录（记录车牌、司机、出发地/目的地、温度），自动上链，更新 batch_status=7
PUT	/api/transport/update	更新运输记录
DELETE	/api/transport/delete/{id}	删除运输记录

上链 traceId 格式：`{batchCode}_TRANSPORT_{transportId}`

5.9 销售记录模块（SaleController）

路径前缀：`/api/sale`

方法	路径	功能
GET	/api/sale/available-quantity	查询批次可销售数量（已运输数量 - 已销售数量）
GET	/api/sale/list	分页查询销售记录

方法	路径	功能
POST	/api/sale/create	创建销售记录（记录销售地点、买家、数量、价格），自动上链，更新 batch_status=8
PUT	/api/sale/update	更新销售记录

上链 traceId 格式：`{batchCode}_SALE_{saleId}`

5.10 溯源查询模块（TraceController）

路径前缀：`/api`

方法	路径	功能
GET	/api/trace/detail	获取溯源详情（聚合批次信息 + 全生命周期时间轴 + 企业信息），同时写入 user_scan_log
POST	/api/blockchain/verify	区块链验真（验证该批次所有环节的数据哈希是否与链上一致）

TraceService 核心逻辑：

`getTraceDetail(batchId, batchCode, openid, ...)`：

1. 按 batchCode 或 id 查询 trace_batch
2. 聚合所有溯源环节：trace_record、trace_inspection、trace_processing、trace_storage、trace_transport、trace_sale
3. 构建时间轴节点列表，每个节点包含：stage（环节标识）、title（环节名称）、time（操作时间）、operator（操作人/企业）、details（详情字段）、images（图片列表）、txHash、blockNumber
4. 写入 user_scan_log（openid、scan_time、ip_address、device_type、user_agent）

`verifyBlockchain(batchId, batchCode)`：

1. 遍历所有有 tx_hash 的环节记录
2. 对每条记录调用 Web3jUtils.verifyHash(traceId, dataHash)，比对链上存储的哈希与本地计算哈希
3. 返回：verified（是否全部通过）、totalNodes（总节点数）、verifiedNodes（验证通过数）、nodeResults（每个节点的验证详情）

各环节 traceId 格式汇总：

环节	traceId 格式
批次初始化	`{batchCode}_INIT`
生长记录	`{batchCode}_RECORD_{recordId}`
检疫质检	`{batchCode}_INSPECTION_{inspectionId}`
加工记录	`{batchCode}_PROC_{processingId}`

环节	traceId 格式
仓储入库	`{batchCode}_STORAGE_{storageId}`
运输记录	`{batchCode}_TRANSPORT_{transportId}`
销售记录	`{batchCode}_SALE_{saleId}`

5.11 区块链模块（BlockchainController + BlockchainContractController）

BlockchainController 路径前缀：`/api/blockchain`

方法	路径	功能
GET	/api/blockchain/stats	查询区块链统计数据（总上链数、今日上链数、Gas 费用等）
GET	/api/blockchain/logs	分页查询上链日志（支持按 enterpriseId、bizType、txHash、batchCode 筛选）
POST	/api/blockchain/retry/{id}	失败记录重新上链
GET	/api/blockchain/node/status	查询以太坊节点连接状态（Ganache 是否在线）

BlockchainContractController 路径前缀：`/api/blockchain/contract`

方法	路径	功能
GET	/api/blockchain/contract/info	查询当前部署的智能合约信息（合约地址、ABI、部署时间）
PUT	/api/blockchain/contract/update	更新合约地址配置

BlockchainTransactionService 核心逻辑：

`recordTransaction(txHash, bizType, bizId, batchId, dataHash, receipt)`：

- 幂等写入：先按 tx_hash 查询 blockchain_transaction 表，已存在则跳过
- 从 TransactionReceipt 中提取 gasUsed、effectiveGasPrice，计算 Gas 费用
- 聚合更新 blockchain_gas_fee 日统计表（按日期累加 gas_used 和 gas_fee）
- bizType 枚举：BATCH / RECORD / INSPECTION / PROCESSING / STORAGE / TRANSPORT / SALE

5.12 数据看板模块（DashboardController）

路径前缀：`/api/dashboard`

方法	路径	功能
GET	/api/dashboard/farmer-stats	养殖企业首页统计

方法	路径	功能
GET	/api/dashboard/processor-stats	加工企业首页统计
GET	/api/dashboard/quarantine-stats	检疫机构首页统计
GET	/api/dashboard/admin-stats	管理员统计大屏
GET	/api/dashboard/analysis-stats	数据统计分析（趋势图数据）

所有统计接口均使用 JdbcTemplate 直接执行 SQL 查询（非 MyBatis-Plus），避免 ORM 层聚合性能损耗。

各端返回字段：

养殖企业（farmer-stats）：activeBatchCount（在养批次数）、totalLivestock（存栏总数）、totalOnChain（累计上链条数）、warningCount（超期预警数）、batchCount（批次总数）、recordCount（记录总数）、dailyTrend（近7天录入趋势）

加工企业（processor-stats）：今日加工批次数、累计加工批次数、累计上链数、加工趋势图、转化率数据

检疫机构（quarantine-stats）：检疫总数、合格数、合格率、上链数、近30天趋势、上链活跃度

管理员大屏（admin-stats）：type1Count（种植养殖企业数）、type2Count（加工宰杀企业数）、type3Count（检疫质检企业数）、totalBatches（全平台批次总数）、totalOnChain（全平台上链总数）、totalUsers（启用用户数）、batchStatusCounts（各状态批次分布）、monthlyBatches（近12个月批次创建趋势）

5.13 系统管理模块

SystemUserController（`/api/system/user`）

方法	路径	功能
GET	/api/system/user/list	分页查询用户列表（支持关键词、用户类型、状态筛选）
POST	/api/system/user/add	新增用户
PUT	/api/system/user/update	更新用户信息
DELETE	/api/system/user/delete/{id}	删除用户
PUT	/api/system/user/reset-password/{id}	重置密码（重置为默认密码）
PUT	/api/system/user/toggle-status/{id}	启用/禁用用户
PUT	/api/system/user/change-password	修改当前用户密码（需验证旧密码）

SystemRoleController（`/api/system/role`）：角色列表、新增角色、更新角色、删除角色、角色菜单权限配置

SystemMenuController（`/api/system/menu`）：菜单树查询、新增/更新/删除菜单

SystemConfigController（`/api/system/config`）：系统配置读写（键值对形式，如系统名称、Logo、联系方式等）

SystemLogController（`/api/system/log`）：操作日志查询（sys_operation_log）、登录日志查询（sys_login_log）

5.14 其他模块

FileController（`/api/file`）：

- 文件上传（multipart/form-data），保存到 `./uploads/` 目录，返回相对路径
- 静态资源通过 `/uploads/**` 路径访问
- FileCleanupTask 定时任务：每天凌晨扫描 uploads 目录，清理超过 7 天且未被任何业务表引用的孤立文件

BannerController（`/api/banner`）：

- 轮播图 CRUD（管理端）
- `GET /api/banner/active`：查询启用中的轮播图，支持按 enterprise_id 过滤（企业专属轮播图）
- target_type 字段：0-所有人（游客）1-种植养殖企业 2-加工宰杀企业 3-检疫质检企业

EduArticleController（`/api/edu/article`）：

- 公开接口（无需登录）：`/published`（已发布文章列表）、`/detail/{id}`（文章详情）、`/view/{id}`（阅读量+1）
- 管理端接口（需登录）：`/list`、`/add`、`/update`、`/delete/{id}`、`/toggle-status/{id}`

MessageController（`/api/message`）：

- 消息查询、标记已读、批量删除、未读数量统计（按 type 分类：system/business）
- 消息发送方式（供其他模块调用）：
  - `sendMessage(userId, ...)` — 发送给指定用户
  - `sendToEnterprise(enterpriseId, ...)` — 发送给企业下所有启用用户
  - `sendToAdmins(...)` — 发送给所有 user_type=1 且 status=1 的管理员
- 消息触发场景：企业入驻申请、审核通过/驳回、批次创建、检疫申请、上链成功、批次接收被拒绝

FeedbackController（`/api/feedback`）：用户反馈提交与管理（查询、回复、删除）

ImageAuditController（`/api/image-audit`）：企业 Logo/封面图变更需管理员审核后生效，审核通过前展示旧图

ScanStatsController（`/api/scan`）：扫码统计数据查询（按批次、时间段、地区维度）

TemplateController（`/api/template`）：溯源模板管理（模板、阶段、字段的 CRUD，供企业自定义溯源录入表单）

5.15 核心工具类与配置

Web3jUtils：

- `init()`：@PostConstruct 初始化，优先使用 Ganache unlocked 账户模式（ClientTransactionManager），回退到私钥签名模式（RawTransactionManager）
- `uploadHash(traceId, dataHash)`：调用智能合约 `uploadHash` 方法，将 traceId 和 SHA-256 哈希写入区块链，返回交易哈希（txHash）
- `getHash(traceId)`：调用合约 `getHash` 方法（只读），返回 BlockchainRecord（traceId、dataHash、operator、timestamp）
- `verifyHash(traceId, dataHash)`：调用 getHash 后比对哈希值，返回 true/false
- `getTransactionReceipt(txHash)`：查询交易回执，获取 blockNumber、gasUsed 等信息

BlockchainTransactionService：

- `recordTransaction(...)`：幂等保存链上交易记录到 blockchain_transaction 表，提取 Gas 信息，聚合更新 blockchain_gas_fee 日统计
- bizType 枚举常量：BIZ_BATCH / BIZ_RECORD / BIZ_INSPECTION / BIZ_PROCESSING / BIZ_STORAGE / BIZ_TRANSPORT / BIZ_SALE

OperationLogAspect：AOP 切面，拦截 @OperationLog 注解的方法，自动记录操作人、操作内容、请求参数、执行结果到 sys_operation_log 表

GlobalExceptionHandler：全局异常处理，捕获 RuntimeException 返回 Result.fail(message)，捕获 MethodArgumentNotValidException 返回参数校验错误详情

CorsConfig：全局跨域配置，允许所有来源（开发环境）

MyBatisPlusConfig：配置分页插件（PaginationInnerInterceptor）

六、微信小程序说明

小程序 AppID：`wxf24401df8316ca97`，导航栏主色：`#4CAF50`（绿色），背景色：`#E8F5E9`

6.1 全局配置（app.js）

globalData 全局状态：

字段	类型	说明
token	String	JWT Token，所有接口请求自动携带
userRole	String	用户角色：admin / enterprise / enterprise-review / consumer / 空字符串（未登录）
enterpriseType	Number	企业类型：1-种植养殖 2-加工宰杀 3-检疫质检，非企业用户为 null
userInfo	Object	当前用户完整信息对象（含 enterpriseId、enterpriseName、auditStatus 等）
apiBaseUrl	String	后端接口基础地址，从 config.js 读取（dev：局域网 IP + 端口 8888；prod：HTTPS 域名）

核心方法：

`onLaunch()`：启动时依次调用 checkLogin() 和 checkUpdate()

`checkLogin()`：从本地缓存（wx.getStorageSync）恢复 token、userInfo、userRole、enterpriseType，恢复后异步调用 validateToken() 验证 Token 有效性

`validateToken()`：请求 `GET /auth/info`，成功则刷新 globalData 和缓存；Token 失效（code != 200）则静默调用 clearLoginInfo()；网络异常时保留本地缓存（不强制登出）

`setLoginInfo(data)`：登录成功后将 token、userInfo、userRole、enterpriseType 同时写入 globalData 和 wx.setStorageSync

`clearLoginInfo()`：清除 globalData 和本地缓存中的所有登录状态

`requireLogin()`：检查 token + userInfo 是否存在，未登录则弹窗引导跳转 `/pages/login/login`，返回 false

`resolveUserRole(userInfo)`：角色推导规则：

- userType=1 → "admin"
- userType=2 且 auditStatus=1 → "enterprise"
- userType=2 且 auditStatus!=1 → "enterprise-review"
- userType=3 → "consumer"

`refreshUserInfo(callback)`：重新请求 `/auth/info` 刷新用户信息，完成后执行回调（各页面 onShow 时调用）

`checkUpdate()`：调用 wx.getUpdateManager 检测小程序更新，有新版本时弹窗提示重启

环境配置（config.js）：

配置项	说明
ENV	当前环境标识（dev/prod），切换环境只需修改此变量
API_BASE_URL	后端接口地址（dev：局域网 IP + 端口 8888；prod：HTTPS 域名）
UPLOAD_URL	文件上传地址（API_BASE_URL + /api/file/upload）
IMAGE_BASE_URL	图片访问前缀，用于拼接 /uploads/... 相对路径为完整 URL
USE_MOCK	Mock 开关，默认 false

请求封装（utils/request.js）：

- 自动携带 `Authorization: Bearer <token>` 请求头（从 globalData 或 Storage 读取）
- GET 请求自动将 data 对象序列化为 URL 查询参数（跳过 null/undefined/空字符串）
- 统一处理 HTTP 401 和业务 code=401：清除登录状态，弹窗跳转登录页
- 支持方法：get / post / put / delete / upload（wx.uploadFile 封装）/ download

表单提交辅助（utils/submit-helper.js）：

- `injectCommonFields(data, options)`：自动注入 enterpriseId、operator（操作人姓名）、日期字段默认值

- `uploadImage(tempFilePath, type)` / `uploadImages(paths, type)`：上传图片到 `/api/file/upload`，已上传的 URL 自动跳过
- 枚举映射：`mapInspectionResult`（合格/不合格 → 1/0）、`mapStorageType`（入库/出库 → 1/2）、`mapProductType`（肉鸡/西红柿 → 1/2）

6.2 底部 TabBar（3 个 Tab）

Tab	页面路径	说明
首页	pages/index/index	平台首页，游客/企业双视图自适应
工作台	pages/workbench/workbench	企业操作入口，按角色展示不同功能模块
我的	pages/user/user	个人中心，展示企业信息和管理入口

TabBar 选中色：`#4CAF50`，未选中色：`#7A7E83`

6.3 首页模块（pages/index/）

index（首页）：

游客视图（未登录）：

- 展示平台品牌宣传、全平台上链总数（调用 `/api/dashboard/farmer-stats`，不传 enterpriseId，取 totalOnChain）
- 展示入驻企业总数（调用 `/api/dashboard/admin-stats`，取 type1Count+type2Count+type3Count 之和）
- 展示平台通用轮播图（调用 `/api/banner/active`，不传 enterpriseId）
- 功能入口：扫码溯源、科普教育、企业信息（跳转企业列表）

企业视图（已登录 userType=2）：

- 展示企业名称、身份标签（认证养殖户/认证加工企业/认证检测机构）、品牌标语
- 展示本企业累计上链数（调用对应 dashboard 接口，传 enterpriseId）
- 展示企业专属轮播图（调用 `/api/banner/active?enterpriseId=xxx`）
- 功能入口：扫码溯源、科普教育、企业信息（跳转本企业详情页）

各企业类型文案差异：

enterpriseType	身份标签	品牌标语	贡献数标签
1（种植养殖）	认证养殖户	科学养殖·品质保证·区块链溯源	本农场累计贡献
2（加工宰杀）	认证加工企业	智能加工·冷链保鲜·区块链溯源	本厂累计加工笔数
3（检疫质检）	认证检测机构	科学检测·权威认证·区块链存证	本机构检测数

enterprise-list（企业列表）：展示所有已审核通过的企业，支持搜索，调用 `/api/enterprise/list`

enterprise-detail（企业详情）：展示企业资质、联系方式、上链统计数据，调用 `/api/enterprise/detail/{id}`

6.4 扫码溯源模块（pages/scan/ + pages/trace/）

scan（扫码页）：

- 调用微信 `wx.scanCode` API 扫描二维码/条形码
- 解析扫码结果，提取批次编号（batchCode），跳转到 trace 页：`/pages/trace/trace?code={batchCode}`

trace（溯源详情页）：

- 接收 URL 参数：batchId / id / code（batchCode），调用 `GET /api/trace/detail` 获取完整溯源数据
- 展示产品基本信息：名称、批次号、生产日期、产品图片、来源企业
- 展示全生命周期时间轴（养殖 → 检疫 → 加工 → 仓储 → 运输 → 销售），每个节点显示：
  - 操作时间、操作人/企业名称
  - 详情字段（因环节不同而异）
  - 现场照片（可点击预览）
  - 区块链信息（txHash、区块高度，可展开查看）
- 检疫节点特殊展示：合格/不合格标签、证书编号、证书图片预览
- 区块链验真按钮：调用 `POST /api/blockchain/verify`，弹窗展示各节点验证状态（通过/失败）
- 企业信息卡片：展示来源企业 Logo、名称，可跳转企业详情页
- 扫码行为自动写入 user_scan_log（openid、IP、设备类型）

6.5 工作台模块（pages/workbench/）

workbench（工作台）：onShow 时调用 `app.refreshUserInfo()` 刷新用户状态，根据角色渲染不同内容：

未登录：显示登录引导按钮

普通用户（consumer，userType=3）：显示"申请企业入驻"引导入口，跳转 `/pages/login/register`

审核中企业（enterprise-review，userType=2 且 auditStatus!=1）：

- 显示当前审核状态（待审核 auditStatus=0 / 已驳回 auditStatus=2）
- 驳回时显示驳回原因，提供"重新提交"按钮，跳转 `/pages/login/register?mode=reapply&enterpriseId=xxx`

已通过企业（enterprise，userType=2 且 auditStatus=1）：

enterpriseType	工作台功能入口	统计数据
1（种植养殖）	批次管理、过程记录、区块链存证	批次数、记录数、上链数（调用 farmer-stats）
2（加工宰杀）	加工记录、仓储管理、运输管理、扫码接收	加工批次数、上链数（调用 processor-stats）

enterpriseType	工作台功能入口	统计数据
3（检疫质检）	检疫申报列表、报告上传	待处理数（pending）、已完成数（completed）（调用 list-for-inspector）

管理员（admin，userType=1）：企业审核、用户管理、区块链管理入口

6.6 批次管理模块（pages/batch/）

batch（批次列表）：

- Tab 配置按企业类型动态切换，调用 `/api/batch/list`（分页）和 `/api/batch/status-count`（各 Tab 数量角标）

养殖企业（enterpriseType=1）Tab：

Tab	标签	状态范围
all	全部	不过滤
breeding	养殖中	1, 2
pending	待检疫	3（橙色角标预警）
completed	已出栏	4, 5, 6, 7, 8, 9

加工企业（enterpriseType=2）Tab：

Tab	标签	状态范围
all	全部	不过滤
pendingReceive	待接收	5, 7（红色角标预警）
processing	加工中	4, 9
finished	已完结	6, 8

检疫机构（enterpriseType=3）Tab：

Tab	标签	状态范围
all	全部	不过滤
pending	待处理	3（红色角标预警）
passed	已合格	5, 6, 7, 8, 9
failed	不合格	不过滤（由 check_result=0 筛选）

管理员（userType=1）Tab：全部 / 养殖期（1,2,3）/ 流通期（4,5,6,7,9）/ 已销售（8）

create（创建批次）：

- 支持两种产品模板：肉鸡养殖（productType=1，单位：只）、西红柿种植（productType=2，单位：株）
- 支持 GPS 定位获取产地坐标（`wx.chooseLocation`），坐标存入 latitude/longitude
- 表单验证后调用 `POST /api/batch/create`，返回 batchCode 和二维码 URL

detail（批次详情）：展示批次完整信息（基本信息 + 所有溯源环节记录列表），调用 `/api/batch/detail`

add-record（添加过程记录）：

- 选择记录类型（feeding/vaccine/inspect/fertilize/irrigate/pesticide）
- 填写操作内容、用药/饲料名称、数量、现场照片（最多6张）
- 调用 `POST /api/record/create`，提交后显示上链交易哈希

record-list（记录列表）：展示某批次的所有过程记录，调用 `/api/record/list`

record-detail（记录详情）：展示单条记录详情，包含区块链存证信息（txHash、区块高度、合约地址）

6.7 数据采集模块（pages/collect/）

各采集页面均通过 submit-helper.js 自动注入 enterpriseId 和 operator，支持图片上传（最多6张），提交后自动上链：

页面	功能	对应接口
process（过程记录）	养殖/种植过程记录采集（记录类型、操作内容、现场照片）	POST /api/record/create
processing（加工记录）	加工宰杀信息采集（工艺、规格、投入/产出数量）	POST /api/processing/create
inspection（检疫申报）	检疫结果录入（检疫员、证书编号、证书图片，check_result 枚举映射）	POST /api/quarantine/submit
storage（仓储入库）	入库信息采集（仓库名称、位置、温湿度，storage_type 枚举映射）	POST /api/storage/create
transport（运输记录）	运输信息采集（车牌、司机、出发地/目的地、温度，指定目标加工企业）	POST /api/transport/create
sale（销售记录）	销售信息采集（销售地点、买家、数量、价格）	POST /api/sale/create
receive（扫码接收）	加工企业扫码接收养殖批次（先调用 quarantine-check 验证检疫合格，再提交接收）	POST /api/batch/receive

receive 页面流程：

1. 扫描批次二维码，获取 batchCode
2. 调用 `GET /api/batch/quarantine-check/{batchCode}` 验证检疫合格状态
3. 展示批次信息和检疫证书，确认后调用 `POST /api/batch/receive`

6.8 检疫模块（pages/quarantine/）

apply-list（申报接收列表）：检疫质检机构专属页面

- 分 Tab：待检疫（status=pending）、已完成（status=completed）
- 调用 `GET /api/quarantine/apply/list-for-inspector`，自动传入当前用户的 enterpriseId 作为 inspectionEnterpriseId
- 列表字段：批次号、产品名称、产品类型标签（养殖类/种植类）、来源企业名称、数量、申报时间、检疫状态
- 点击卡片或"去处理"按钮跳转 result-form 页，URL 携带 batchCode、productName、quantity、unit、enterpriseName、batchId

result-form（检疫结果录入）：

- 接收 URL 参数展示批次基本信息（只读）
- 填写：检疫日期、检疫员姓名、检疫项目、检疫结果（合格/不合格，映射为 1/0）、证书编号
- 上传检疫证书图片（调用 `/api/file/upload`）
- 提交调用 `POST /api/quarantine/submit`，成功后显示上链交易哈希（txHash）

report-upload（报告上传）：上传检疫报告文件，调用 `POST /api/quarantine/update-certificate`

6.9 区块链模块（pages/blockchain/）

certificate（存证证书）：

- 接收 URL 参数：txHash（优先）或 batchNo
- 调用 `GET /api/blockchain/logs?txHash=xxx` 或 `?batchCode=xxx` 查询链上记录
- 展示数字证书卡片：产品名称、批次号、企业名称、记录类型名称（feeding/vaccine/inspect 等映射为中文）、交易哈希、区块高度、合约地址、存证时间
- 支持复制哈希值（wx.setClipboardData）、分享证书（onShareAppMessage）

记录类型映射：

recordType	中文名称	证书标题
init	批次初始化	批次创建存证
feeding	喂养记录	喂养作业存证
vaccine	防疫记录	防疫接种存证
inspect	环境巡查	巡查记录存证
quarantine	检疫申报	检疫申报存证
storage	仓储入库	仓储入库存证
transport	物流运输	物流运输存证

record-list（上链记录列表）：

- 展示企业所有上链记录，调用 `GET /api/blockchain/logs`（传 enterpriseId）
- 支持按业务类型筛选，显示上链状态、Gas 费用、交易哈希
- 点击记录跳转 certificate 页查看数字证书

6.10 管理员模块（pages/admin/）

enterprise-list（企业管理）：

- 统计卡片：总数/待审核/已通过/已驳回（调用 `/api/enterprise/list` 统计）
- 支持关键词搜索、审核状态筛选、企业类型筛选
- 操作：查看资质图片（营业执照/生产许可证预览）、通过审核、驳回审核（填写驳回原因）
- 审核操作调用 `POST /api/enterprise/audit/approve`

user-list（用户管理）：

- 查询所有用户，支持关键词搜索、用户类型筛选
- 调用 `GET /api/system/user/list`
- 点击用户跳转 user-detail 页

user-detail（用户详情）：

- 展示用户基本信息、关联企业信息
- 支持修改用户状态（启用/禁用）、重置密码

blockchain（区块链管理）：

- 展示节点连接状态（调用 `GET /api/blockchain/node/status`）
- 展示上链统计：总上链数、今日上链数、Gas 费用（调用 `GET /api/blockchain/stats`）

6.11 用户中心模块（pages/user/）

user（我的）：onShow 时调用 `app.refreshUserInfo()` 刷新状态，根据角色渲染：

未登录：显示登录/注册入口

普通用户（userType=3）：显示用户名、角色名称（普通用户）、消息入口

企业用户（userType=2）：

- 展示企业 Logo（http 图片通过 util.proxyImage 转为本地临时文件解决真机加载问题）
- 展示企业名称、企业识别码（可复制）、联系电话、地址（省市区+详细地址拼接）
- 展示营业执照/生产许可证图片（可点击预览）
- 展示累计上链记录数（调用 `/api/blockchain/stats` 取 totalCount）
- 支持更换 Logo：上传图片后调用 `POST /api/image-audit/submit` 提交审核，管理员审核通过后生效
- 数据来源：`GET /api/enterprise/detail/{enterpriseId}`

管理员（userType=1）：显示企业审核、用户管理、区块链管理快捷入口

register（企业入驻注册）：

- 填写企业基本信息：名称、统一社会信用代码、企业类型、法人姓名、联系人、联系方式、地址
- 上传营业执照、生产许可证图片
- 支持 reapply 模式（URL 参数 mode=reapply&enterpriseId=xxx），被驳回后重新提交
- 调用 `POST /api/enterprise/register`

manage（企业信息管理）：编辑企业信息，调用 `PUT /api/enterprise/update`

messages（消息列表）：

- 分 Tab：全部、系统通知（type=system）、业务提醒（type=business）
- 调用 `GET /api/message/list`，显示未读数角标
- 支持批量标记已读、批量删除

message-detail（消息详情）：展示消息完整内容，自动标记已读（调用 `PUT /api/message/read`）

profile（个人资料）：修改密码，调用 `PUT /api/system/user/change-password`（需验证旧密码）

6.12 登录注册模块（pages/login/）

login（登录页）：

- 用户名/密码登录，调用 `POST /auth/login`
- 登录成功后调用 `app.setLoginInfo()` 保存 token、userInfo、userRole、enterpriseType
- 根据 userRole 跳转：admin/enterprise → 首页（switchTab），其他 → 首页
- 登录失败显示错误提示（Toast）

register（企业入驻注册页）：

- 普通用户注册：调用 `POST /auth/register`，创建 userType=3 账号
- 企业入驻注册：调用 `POST /api/enterprise/register`，填写企业信息并上传资质图片

6.13 科普教育模块（pages/education/）

education（科普文章列表）：

- 展示科普文章列表，支持按分类筛选（farming/vaccine/process/hygiene/quarantine/blockchain/safety）
- 调用 `GET /api/edu/article/published`，支持按 enterpriseId、category 过滤
- 点击文章跳转详情页（调用 `/api/edu/article/detail/{id}`），阅读时自动调用 `/api/edu/article/view/{id}` 增加阅读量

6.14 企业公示模块（pages/enterprise/）

enterprise（企业公示）：展示企业公开信息（资质、联系方式、上链统计），调用 `/api/enterprise/detail/{id}`

audit（企业审核详情）：管理员审核企业入驻申请，查看资质图片，执行通过/驳回操作，调用 `POST /api/enterprise/audit/approve`

license（资质证书）：展示企业营业执照、生产许可证等资质图片（wx.previewImage 预览）

七、区块链集成说明

7.1 智能合约（Traceability.sol）

合约部署在 Ganache 本地以太坊测试网（chainId: 1337，RPC: http://127.0.0.1:8545）。

核心函数：

- `uploadHash(string traceId, string dataHash) returns (bool)`：将溯源ID和数据哈希写入链上存储，记录操作者地址和时间戳
- `getHash(string traceId) returns (string, string, address, uint256)`：查询链上记录，返回 traceId、dataHash、operator、timestamp

7.2 数据上链流程

每个业务环节（批次创建、过程记录、检疫、加工、仓储、运输、销售）在数据写入 MySQL 后，执行以下上链流程：

1. 将业务数据序列化为 JSON 字符串
2. 计算 SHA-256 哈希值（dataHash）
3. 以 `业务类型_业务ID` 为 traceId，调用 `Web3jUtils.uploadHash(traceId, dataHash)`
4. 获取交易哈希（txHash），等待交易回执获取区块号（blockNumber）
5. 将 txHash、blockNumber、chainTime、dataHash 回写到对应业务表
6. 调用 `BlockchainTransactionService.recordTransaction()` 保存交易记录，更新日 Gas 统计

7.3 溯源验真流程

消费者扫码后，在溯源详情页点击"区块链验真"：

1. 前端调用 `POST /api/blockchain/verify`，传入 batchId
2. 后端查询该批次所有有 txHash 的环节记录
3. 对每个环节，重新计算当前数据库数据的 SHA-256 哈希
4. 调用 `Web3jUtils.verifyHash(traceId, currentDataHash)` 与链上哈希比对
5. 返回每个节点的验证结果（verified/txHash/nodeName）
6. 全部节点验证通过则 verified=true，表示数据未被篡改

7.4 区块链配置项（application.yml）

```yaml
blockchain:
  rpc-url: http://127.0.0.1:8545        # Ganache RPC 地址
  contract-address: 0x...               # 已部署的合约地址（必须配置）
  account-address: 0x...                # Ganache 解锁账户地址（开发环境）
  gas-limit: 3000000                    # Gas 上限
  gas-price: 20                         # Gas 价格（Gwei）
```

八、系统运行说明

8.1 环境要求

软件	版本
JDK	11
Maven	3.6+
MySQL	8.0
Ganache	2.7+（桌面版）
微信开发者工具	最新稳定版
Node.js	16.x LTS（PC 前端）

8.2 启动顺序

1. 启动 Ganache（必须先于后端启动，RPC 地址 http://127.0.0.1:8545）
2. 在 Remix IDE 部署 `05_SmartContract/Traceability.sol`，获取合约地址
3. 修改 `02_Backend/src/main/resources/application.yml`：填写 `contract-address` 和 `account-address`
4. 初始化数据库：`mysql -u root -p agricultural < 01_Database/agricultural_database.sql`
5. 启动后端：在 `02_Backend` 目录下执行 `mvn spring-boot:run`（端口 8888）
6. 启动 PC 前端：在 `03_Frontend_Web` 目录下执行 `npm install`（首次运行），再执行 `npm run dev`（端口 8080）
7. 配置小程序：修改 `04_Frontend_MiniApp/config.js` 中的 `API_BASE_URL` 为本机局域网 IP，在微信开发者工具导入 `04_Frontend_MiniApp` 目录

8.3 默认账号

角色	用户名	密码
平台管理员	admin	admin123

8.4 注入演示数据（可选）

```
cd 02_Backend
mvn spring-boot:run "-Dspring-boot.run.profiles=seed"
```

注入 100 个肉鸡批次及约 2000 条溯源记录，并同步上链（约 2-3 分钟）。注入完成后需重新以正常模式启动后端。

8.5 关键配置文件速查

文件	关键配置
`02_Backend/src/main/resources/application.yml`	数据库密码、区块链合约地址、账户地址、文件上传目录
`04_Frontend_MiniApp/config.js`	后端 API 地址（dev/prod 环境切换）
`04_Frontend_MiniApp/project.config.json`	小程序 AppID
`03_Frontend_Web/.env.production`	PC 前端生产环境后端地址

8.6 常见问题

现象	原因	解决方案
后端启动报 `Connection refused: 127.0.0.1:8545`	Ganache 未启动	先启动 Ganache
后端启动报 `Access denied for user 'root'`	MySQL 密码错误	修改 application.yml 中 datasource.password
上链操作全部失败	合约地址配置错误或 Ganache 重启	重新部署合约，更新 contract-address
小程序报 `request:fail`	IP 地址错误或未关闭域名校验	检查 config.js 中 IP，勾选"不校验合法域名"
小程序图片不显示	IMAGE_BASE_URL 未正确拼接	确认 config.js 中 API_BASE_URL 填写正确
小程序正式版接口失败	未配置服务器域名或未使用 HTTPS	在微信公众平台添加合法域名

九、用户角色与权限说明

9.1 角色权限总览

角色	userType	说明	可访问功能
平台管理员	1	系统超级管理员	全部功能，含企业审核、用户管理、系统配置、区块链管理
种植养殖企业	2（enterpriseType=1）	审核通过的养殖/种植企业	批次管理、过程记录、检疫申请、区块链存证
加工宰杀企业	2（enterpriseType=2）	审核通过的加工企业	扫码接收批次、加工记录、仓储管理、运输管理、销售记录
检疫质检企业	2（enterpriseType=3）	审核通过的检疫机构	检疫申报列表、检疫结果录入、报告上传

角色	userType	说明	可访问功能
审核中企业	2（auditStatus!=1）	已提交入驻申请但未通过	查看审核状态，驳回后可重新提交
普通用户/消费者	3	注册用户或游客	扫码溯源、查看溯源详情、区块链验真、科普教育、企业公示

9.2 角色关系说明

本系统共有 5 类用户身份，每类身份对应不同的操作权限和小程序界面。

一、身份是怎么来的?

用户身份由注册/审核流程决定，分三条路径：

```
路径 A：直接注册普通用户
  填写用户名密码注册 -> 身份：消费者
  可以：扫码查看溯源信息，无法进行任何业务操作

路径 B：企业入驻申请
  提交企业资料 -> 等待管理员审核 -> 审核通过
  审核通过后身份自动升级为对应企业类型，解锁业务功能

路径 C：平台管理员
  数据库初始化时预置，不通过注册流程创建
```

二、五类身份及其权限

身份	如何获得	小程序工作台显示内容
消费者	普通注册	无工作台，仅可扫码溯源
审核中企业	已提交入驻申请，尚未通过	显示审核进度，被驳回可重新提交
种植养殖企业	企业入驻审核通过（类型=养殖/种植）	批次管理、过程记录、检疫申请、区块链存证
加工宰杀企业	企业入驻审核通过（类型=加工宰杀）	扫码接收批次、加工记录、仓储、运输、销售
检疫质检企业	企业入驻审核通过（类型=检疫质检）	检疫申报列表、检疫结果录入、报告上传
平台管理员	数据库预置	企业审核、用户管理、系统配置、区块链管理

三、企业审核通过时系统自动完成的操作

管理员点击"审核通过"后，后端自动执行以下三步，无需手动操作：

1. 将企业状态标记为"已通过"
2. 将用户账号与该企业绑定，同步企业类型（种植养殖/加工宰杀/检疫质检）
3. 为用户分配对应角色，解锁相应业务功能